SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

---

COURSE OUTCOME COVERED

CO5: *Develop the network security strategies based on the study of viruses, worms, firewall architectures, and IDS/IPS functionalities.CO* (BL5 , BL6)

Assessment Tool Weightage: 20%

QUESTIONS: 5 **TOTAL MARKS: 25**

Annexure A
PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Code of Conduct:

I S. Harsha Vardhan Reddy (192325029) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
S. Harsha Vardhan Reddy

Annexure A

PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

1. Design pseudocode for a **stateful firewall system** that tracks active connections and filters packets based on session state. Evaluate how state tracking improves security compared to stateless filtering.

---

2. Design pseudocode for a **signature-based malware detection system** that scans files and compares them against a known signature database. Evaluate its effectiveness in detecting known threats and its limitations against new attacks.

---

3. Design pseudocode for a **distributed denial-of-service (DDoS) detection system** using traffic threshold analysis. Evaluate how the algorithm differentiates between legitimate high traffic and attack traffic.

---

4. Design pseudocode for a **network intrusion detection system using machine learning classification**. Evaluate how feature selection and training data impact detection accuracy.

---

5. Design pseudocode for a **secure network access control system** that authenticates users and enforces role-based access policies. Evaluate how access control mechanisms enhance overall network security.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

PSEUDOCODE WRITING TASK — ALGORITHM DESIGN

1. Stateful Firewall System

Aim

To design a stateful firewall that tracks active network connections and filters packets according to their session state.

Pseudocode

```
ALGORITHM StatefulFirewall

INPUT:
   Incoming packet P
   Connection State Table

BEGIN

   Extract:
      Source_IP
      Destination_IP
      Source_Port
      Destination_Port
      Protocol
      TCP_Flags

   Connection_ID ←
      (Source_IP, Destination_IP,
       Source_Port, Destination_Port, Protocol)

   IF Connection_ID exists in State_Table THEN

      State ← State_Table[Connection_ID]

      IF Packet is valid for State THEN
         FORWARD packet
         UPDATE connection state
      ELSE
         DROP packet
         LOG "Invalid state packet"
      END IF

   ELSE

      IF P is a new connection request THEN

         IF Firewall_Policy_Allows(P) THEN
            CREATE new connection entry
```

```
            ADD Connection_ID to State_Table
            FORWARD packet
        ELSE
            DROP packet
            LOG "New connection blocked"
        END IF

    ELSE
        DROP packet
        LOG "Unsolicited packet"
    END IF

END IF

FOR each connection in State_Table DO
    IF connection has timed out THEN
        REMOVE connection
    END IF
END FOR

END
```

Evaluation

A stateless firewall examines each packet independently. It does not know whether a packet belongs to an established connection.

A stateful firewall maintains information about active sessions. Therefore, it can:

- Accept packets belonging to legitimate established sessions.
- Reject unsolicited packets.
- Detect invalid TCP states.
- Reduce spoofing and session-abuse opportunities.
- Automatically remove inactive connections using timeouts.

Conclusion

State tracking improves security because the firewall understands the **context of communication**, rather than evaluating packets individually. The main trade-off is additional memory and processing overhead for maintaining the connection-state table.

---

2. Signature-Based Malware Detection System

Aim

To design a malware detection algorithm that scans files and compares their signatures with a database of known malicious signatures.

Pseudocode

```
ALGORITHM SignatureBasedMalwareDetection

INPUT:
    File F
```

```
   Known_Malware_Signature_Database

OUTPUT:
   Detection Result

BEGIN

   IF File F does not exist THEN
      RETURN "File Not Found"
   END IF

   Read file F

   IF File is empty THEN
      RETURN "No Threat Detected"
   END IF

   Signature ← GenerateHash(F)

   IF Signature exists in
     Known_Malware_Signature_Database THEN

      RESULT ← "MALWARE DETECTED"

      QUARANTINE F
      LOG detection details
      ALERT Security Administrator

   ELSE

      RESULT ← "NO KNOWN MALWARE DETECTED"

   END IF

   RETURN RESULT

END
```

Evaluation

The system is highly effective against **known malware** because the malware's signature is already present in the database.

For example:

File → Hash Generation → Signature Database

↓

Match Found?

/ \
YES NO
↓ ↓
Malware No Known Threat

Advantages

- Fast detection.
- Simple implementation.
- Low computational overhead.
- Low false-positive rate when signatures are accurate.
- Effective against previously identified malware.

Limitations

It cannot reliably detect:

- New zero-day malware.
- Previously unknown malware.
- Modified malware with a different signature.
- Polymorphic malware.
- Malware whose signature has been deliberately changed.

Conclusion

Signature-based detection is excellent for **known threats**, but it should be combined with behavioral or anomaly-based detection to identify new and evolving attacks.

---

3. DDoS Detection Using Traffic Threshold Analysis

Aim

To detect possible DDoS attacks by monitoring traffic volume and comparing it with dynamically calculated thresholds.

Pseudocode

```
ALGORITHM DDoSDetection

INPUT:
   Network traffic
   Baseline traffic statistics

OUTPUT:
   Traffic status

BEGIN

   Set Monitoring_Window ← 10 seconds
   Set Alert_Threshold ← calculated baseline + tolerance

   WHILE Network is Active DO

      Packet_Count ← 0
      Unique_Sources ← 0
```

```
Connection_Count ← 0

START Monitoring_Window

WHILE Monitoring_Window is Active DO

   Receive Packet P

   Packet_Count ← Packet_Count + 1

   Record P.Source_IP

   IF P creates a new connection THEN
      Connection_Count ← Connection_Count + 1
   END IF

END WHILE

Unique_Sources ← CountUniqueSourceIPs()

Traffic_Rate ← Packet_Count /
               Monitoring_Window

IF Traffic_Rate > Alert_Threshold THEN

   IF Connection_Count is extremely high
     AND
     traffic pattern is abnormal THEN

      STATUS ← "POSSIBLE DDoS ATTACK"

      Activate_Rate_Limiting
      Block_High_Risk_Sources
      Send_Alert

   ELSE

      STATUS ← "LEGITIMATE HIGH TRAFFIC"

      Continue monitoring

   END IF

ELSE
```

```
        STATUS ← "NORMAL TRAFFIC"

    END IF

    Update_Baseline_Statistics()

  END WHILE

END
```

Differentiating Legitimate High Traffic from DDoS

A simple packet threshold alone is insufficient.

For example, a legitimate event such as an online sale may generate very high traffic. Therefore, the algorithm should consider multiple indicators:

Indicator	Legitimate Traffic	DDoS Traffic
Traffic volume	High	Extremely high
Source behavior	Usually meaningful	Often abnormal
Connection pattern	Normal	Excessive
Request rate	Relatively controlled	Abnormally high
Geographic distribution	Expected	Often suspicious/anomalous
Request validity	Mostly valid	Many malformed/repetitive requests

Conclusion

Threshold analysis provides fast DDoS detection, but static thresholds can produce false positives. **Dynamic thresholds combined with source diversity, connection rate and traffic-pattern analysis** provide better discrimination between legitimate traffic spikes and attacks.

---

4. Network Intrusion Detection Using Machine Learning Classification

Aim

To design an ML-based network IDS that classifies network traffic as normal or malicious.

Pseudocode

```
ALGORITHM ML_Network_IDS

INPUT:
  Historical_Network_Dataset
  Live_Network_Traffic

OUTPUT:
  Traffic Classification

BEGIN

  Load Historical_Network_Dataset
```

Remove missing or invalid records

Clean and normalize dataset

Extract network features:
  Source_IP
  Destination_IP
  Source_Port
  Destination_Port
  Protocol
  Packet_Size
  Connection_Duration
  Packet_Rate
  Byte_Rate

Select important features

Split dataset into:
  Training_Set
  Validation_Set
  Testing_Set

Train ML_Classifier using Training_Set

Evaluate classifier using Validation_Set

IF Accuracy and Detection_Performance
  are acceptable THEN

  Deploy classifier

ELSE

  Modify features
  Tune model parameters
  Retrain classifier

END IF

WHILE Network is Active DO

  Capture network traffic

```
        Extract selected features

        Normalize features

        Prediction ← ML_Classifier(features)

        IF Prediction = "ATTACK" THEN
           Generate Alert
           Log Traffic
           Apply Security Response
        ELSE
           Allow Traffic
        END IF

    END WHILE

END
```

Feature Selection Evaluation

Feature selection is important because irrelevant features can:

- Increase processing time.
- Increase model complexity.
- Introduce noise.
- Reduce classification accuracy.

Useful features may include:

- Packet rate.
- Connection duration.
- Protocol.
- Port numbers.
- Packet size.
- Number of connections.
- Bytes transferred.

Removing unnecessary features can make the model faster and sometimes improve generalization.

Training Data Evaluation

The quality of training data strongly affects IDS performance.

If the dataset contains mostly normal traffic:

Model → biased toward normal traffic

→ attacks may be missed

If the dataset contains outdated or unrealistic attack patterns:

Model → poor performance on modern attacks

Therefore, training data should contain:

- Normal traffic.
- Multiple attack categories.
- Sufficient samples.
- Representative network behavior.

- Properly labeled records.

Conclusion

An ML-based IDS can identify complex attack patterns that are difficult to detect using fixed signatures. However, its effectiveness depends heavily on **feature selection, dataset quality, class balance and continuous model evaluation**.

---

5. Secure Network Access Control with Role-Based Access Control

Aim

To design a secure network access-control system that authenticates users and grants permissions according to their assigned roles.

Pseudocode

```
ALGORITHM RoleBasedNetworkAccessControl

INPUT:
   Username
   Password
   Requested_Resource
   Requested_Action

OUTPUT:
   Access Decision

BEGIN

   RECEIVE Username and Password

   User ← FindUser(Username)

   IF User does not exist THEN
      DENY access
      LOG "Unknown user"
      RETURN
   END IF

   IF Authenticate(User, Password) = FALSE THEN

      DENY access
      LOG "Authentication failed"

      IF FailedAttempts(User) > MaximumAttempts THEN
         LOCK account
      END IF

      RETURN
```

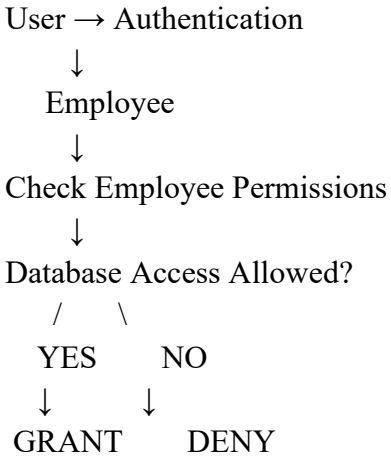
```
    END IF

    Role ← GetUserRole(User)

    IF Role does not exist THEN
        DENY access
        LOG "No role assigned"
        RETURN
    END IF

    Permissions ← GetRolePermissions(Role)

    IF Requested_Resource is in Permissions
      AND Requested_Action is allowed THEN

        GRANT access
        LOG "Access granted"

    ELSE

        DENY access
        LOG "Access denied"

    END IF

END
```

Example Role Structure

```
Administrator
   ↓
Full system management

Manager
   ↓
Reports + department resources

Employee
   ↓
Authorized business applications

Guest
   ↓
Limited network access
```

Example

Suppose an employee requests access to a confidential database.



If the employee's role does not have database permissions, access is denied even when the username and password are correct.

Security Benefits

Role-based access control provides:

- **Authentication:** verifies user identity.
- **Authorization:** determines what the user can access.
- **Least privilege:** users receive only required permissions.
- **Centralized policy management:** permissions are assigned to roles instead of individual users.
- **Accountability:** access attempts can be logged.
- **Reduced insider risk:** unauthorized resource access is restricted.

Conclusion

A combination of strong authentication and **Role-Based Access Control (RBAC)** significantly improves network security. Authentication establishes *who* the user is, while authorization determines *what* that user is allowed to do. This reduces unauthorized access and supports the principle of least privilege.